CodeSignal Cheat Sheet — Exam Day

First 2 Minutes: Read All 6 Levels Before Writing Anything

The Storage Schema (use this from Level 1, don't change it later)

```
from collections import defaultdict
import copy, bisect, threading

class InMemoryDB:
    def __init__(self):
        self._store = defaultdict(dict)
        # _store[key][field] = {"value": "...", "expires_at": float('inf')}
        self._backups = []          # list of (timestamp, deepcopy of _store)
        self._lock = threading.RLock()  # RLock so methods can call each other
```

Alive Check — ALWAYS use this helper

```
def _is_alive(self, entry, timestamp):
    return entry["expires_at"] > timestamp  # strict > (half-open interval)
```

Level 1 — Basic CRUD

```
def set_at(self, key, field, value, timestamp, ttl=None):
    expires = float('inf') if ttl is None else timestamp + ttl
    self._store[key][field] = {"value": value, "expires_at": expires}

def get_at(self, key, field, timestamp):
    entry = self._store.get(key, {}).get(field)
    if entry and self._is_alive(entry, timestamp):
        return entry["value"]
    return None

def delete_at(self, key, field, timestamp):
    entry = self._store.get(key, {}).get(field)
    if entry and self._is_alive(entry, timestamp):
        del self._store[key][field]
        return True
    return False          # ← False, not None

def get_all_keys(self, timestamp):
    # only keys that have at least one alive field
    return [
        key for key, fields in self._store.items()
        if any(self._is_alive(e, timestamp) for e in fields.values())
    ]
```

Level 2 — Scans

```
def scan_at(self, key, timestamp):
    fields = self._store.get(key, {})
    alive = [(f, e) for f, e in fields.items() if self._is_alive(e, timestamp)]
    return [f"{f}({e['value']})" for f, e in sorted(alive)]  # sort by field name

def scan_by_prefix_at(self, key, timestamp, prefix):
    fields = self._store.get(key, {})
    alive = [(f, e) for f, e in fields.items()
             if f.startswith(prefix) and self._is_alive(e, timestamp)]
    return [f"{f}({e['value']})" for f, e in sorted(alive)]
```

Output format: `"field(value)"` — no spaces, sorted by field name, not value.

Level 3 — TTL Semantics

- Set at `t=5`, `ttl=10` → `expires_at = 15`
- Alive at `t=14` ✓ — expired at `t=15` ✗
- No TTL → `expires_at = float('inf')`
- TTL is per `(key, field)` — not per key

Level 4 — Backup & Restore

```
def backup(self, timestamp):
    snapshot = copy.deepcopy(self._store)
    self._backups.append((timestamp, snapshot))
    # count keys with ≥1 alive field (record count, NOT field count)
    count = 0
    for fields in snapshot.values():
        if any(self._is_alive(e, timestamp) for e in fields.values()):
            count += 1
    return count

def restore(self, timestamp, timestamp_to_restore):
    times = [b[0] for b in self._backups]
    idx = bisect.bisect_right(times, timestamp_to_restore) - 1
    if idx < 0:
        return  # no backup found → no-op
    _, snapshot = self._backups[idx]
    new_store = defaultdict(dict)
    for key, fields in snapshot.items():
        for field, entry in fields.items():
            if not self._is_alive(entry, timestamp_to_restore):
                continue  # drop fields already expired at backup time
            if entry["expires_at"] == float('inf'):
                new_expires = float('inf')
            else:
                remaining = entry["expires_at"] - timestamp_to_restore
                new_expires = timestamp + remaining
            new_store[key][field] = {"value": entry["value"], "expires_at": new_expires}
    self._store = new_store
```

TTL rebase formula:

```
remaining = expires_at_at_backup - timestamp_to_restore
new_expires_at = timestamp + remaining
```

Level 5 — Thread Safety

```
# Use RLock (re-entrant) so methods can call each other safely
self._lock = threading.RLock()

def set_at(self, key, field, value, timestamp, ttl=None):
    with self._lock:
        ...  # existing logic

# Wrap EVERY public method with self._lock
```

RLock vs Lock: Use `RLock` if any method calls another method on `self`. Same thread can acquire it twice without deadlock.

Level 6 — Advanced Concurrency (prepare for any of these)

```
# Batch execute
def execute_many(self, operations):
    results = []
    for op_name, *args in operations:
        results.append(getattr(self, op_name)(*args))
    return results

# asyncio version — same logic, add async/await
import asyncio
self._lock = asyncio.Lock()
async def set_at(self, ...):
    async with self._lock:
        ...

# Read-write lock (multiple readers, exclusive writer)
import threading
class RWLock:
    def __init__(self):
        self._readers = 0
        self._r = threading.Lock()
        self._w = threading.Lock()
    def r_acquire(self):
        with self._r:
            self._readers += 1
            if self._readers == 1: self._w.acquire()
    def r_release(self):
        with self._r:
            self._readers -= 1
            if self._readers == 0: self._w.release()
    def w_acquire(self): self._w.acquire()
    def w_release(self): self._w.release()
```

Quick Reference

Thing	Pattern
No TTL	`expires_at = float('inf')`
With TTL	`expires_at = timestamp + ttl`
Alive check	`entry["expires_at"] > timestamp`
Scan item format	`f"{field}({entry['value']})"`
Scan sort	`sorted(alive)` — sorts by field name (tuple first element)
delete missing	return `False` (not `None`)
backup count	records (keys) with ≥1 alive field
restore no-op	if no backup at or before `timestamp_to_restore`
deep copy	`copy.deepcopy(self._store)`
binary search	`bisect.bisect_right(times, t) - 1`
thread lock	`threading.RLock()` + `with self._lock:`

Dictionaries

```
# Create
d = {}
d = {"key": "value"}
from collections import defaultdict
d = defaultdict(dict)   # d[missing_key] auto-creates {}

# Add / update
d["key"] = "value"

# Read
d["key"]               # KeyError if missing
d.get("key")           # None if missing
d.get("key", "default")

# Delete
del d["key"]           # KeyError if missing
d.pop("key", None)     # safe delete, returns value or None

# Check membership
"key" in d
"key" not in d

# Iterate
for k in d:                      # keys
for k, v in d.items():           # key-value pairs
for v in d.values()              # values

# Length
len(d)

# Nested dict access (safe)
d.get("key", {}).get("field")    # None if either missing
```

```
# Dict comprehension
{k: v for k, v in d.items() if v > 0}
```

Lists

```
# Create
lst = []
lst = [1, 2, 3]

# Add
lst.append(x)           # add to end
lst.insert(0, x)        # insert at index

# Remove
lst.pop()               # remove + return last
lst.pop(0)              # remove + return at index
lst.remove(x)           # remove first occurrence (ValueError if missing)
del lst[0]              # remove at index

# Access
lst[0]                  # first
lst[-1]                 # last
lst[1:3]                # slice (index 1 and 2)

# Check membership
x in lst
x not in lst

# Iterate
for item in lst:
for i, item in enumerate(lst):

# Sort
lst.sort()              # in-place, ascending
lst.sort(reverse=True)
sorted(lst)             # returns new list, original unchanged
sorted(lst, key=lambda x: x[0])  # sort by first element of tuple

# Length
len(lst)

# List comprehension
[x * 2 for x in lst]
[x for x in lst if x > 0]
[(f, e) for f, e in d.items() if e["val"] > 0]  # filter dict into list of tuples
```

Common Typos to Avoid

Wrong	Right
`startsWith`	`startswith`
`lst.add(x)`	`lst.append(x)`
`self.store(key)`	`self.store[key]`
`for key in self`	`for key in d`

Wrong	Right
`async` (alone)	`async def`
`ttl:None`	`ttl=None`

Reading a Unittest in 5 Seconds

```
class TestDB(unittest.TestCase):

    def setUp(self):              # 1. ignore this — just creates the object
        self.db = InMemoryDB()

    def test_get_missing_key(self):   # 2. read the test name first — tells you what's being tested
        result = self.db.get("k", "f")
        self.assertIsNone(result)     # 3. last line = the requirement (None if missing)
```

- Test name → what feature is being tested
- Last `assert` line → what your code must return/do
- `setUp` → just boilerplate, skip it

Time Management

- **Read all 6 levels before writing a single line** (2 min) — design Level 1 to handle Level 5/6
- **Run tests after every method**, not after every level
- **Stuck > 10 min?** Move to next level and come back — partial credit is real
- **4 levels beats 3 perfect levels** — done is better than perfect

Gotchas

1. `delete` returns **`False`** for missing/expired — never `None`
2. Expired AT `expires_at` — use `>` not `>=`
3. `backup` counts **records**, not fields
4. On restore: drop fields expired at backup time, rebase remaining TTLs
5. `float('inf') - anything` is still `float('inf')` — no special case needed... but check the prompt
6. Use `RLock` not `Lock` if methods call other methods
7. `defaultdict(dict)` means `self._store[new_key]` auto-creates `{}` — fine for set, but `get` should use `.get(key, {})` to avoid creating empty entries
8. Dict uses `:` not `=` → `{"key": value}` not `{"key" = value}`
9. `del d[key]` actually deletes — `d[key] = None` does NOT delete, it sets to None
10. Always check `key not in d` before `del d[key]` — or it crashes with KeyError
11. `expires_at = None` means no expiry in simple stores — but `float('inf')` is better: lets you always compare without None checks